

SystemC Modeling of a Parameterized Sequential Restoring Divider

Sara Eskandari

School of Electrical and Computer Engineering, college of engineering University of Tehran

sara.esk79@gmail.COM

Abstract—This document details the design and implementation of a sequential restoring division hardware architecture utilizing SystemC. The primary objective is to transition digital design concepts from traditional hardware description languages to a higher level of abstraction using a system-level design approach. A fully parameterized division unit is developed, featuring a controller based on the Huffman coding style to coordinate the multi-cycle execution flow. The architecture strictly separates combinational logic from sequential state registers to ensure reliable hardware synthesis. The project progresses through distinct phases, extending the basic architecture to handle signed operands and elevating the simulation environment to Bus-Functional Modeling.

Keywords— SystemC, Register Transfer Level (RTL), Restoring Division, Parameterized Hardware, Huffman Controller, Finite State Machine (FSM).

I. INTRODUCTION

SystemC provides a robust C++ framework that addresses challenge by enabling designers to model hardware components from the system level down to the Register Transfer Level (RTL) within a unified environment.

This report outlines the development of a parameterized sequential restoring divider using SystemC. The architecture is implemented across multiple distinct phases, beginning with a fundamental RTL design for unsigned integers. To guarantee design scalability, all datapath and control modules are constructed using C++ templates. Furthermore, the physical execution flow is managed by a dedicated controller designed under the Huffman coding style. This approach explicitly separates the next-state combinational logic, sequential state registers, and combinational output decode structures. Following the initial RTL implementation, the project shifts its architectural focus toward higher-level abstraction through Bus-Functional Modeling (BFM).

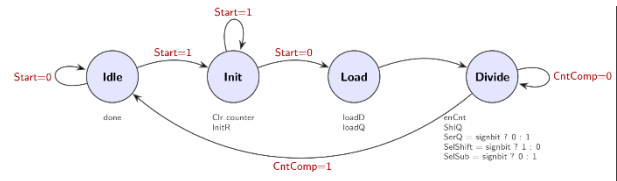
II. DESIGN ARCHITECTURE

A. Design of Controller Unsigned Restoring Division

The first phase focuses on the structural RTL implementation of a sequential unsigned division unit based on the classical restoring algorithm. A core requirement of this design is full

scalability. To achieve this without altering the underlying logic, C++ templates are employed.

The control mechanism is governed by a dedicated Finite State Machine (FSM) modeled strictly following the Huffman coding style.: the combinational next-state logic, the sequential state register update, and the combinational output decoder.



```
1 #include <systemc.h>
2 SC_MODULE(controller) {
3
4     sc_in<sc_logic> clk, rst, start;
5     sc_in<sc_logic> cnt_comp, signbit;
6
7     sc_out<sc_logic> done;
8     sc_out<sc_logic> clr_counter, init_r;
9     sc_out<sc_logic> load_d, load_q, load_r;
10    sc_out<sc_logic> en_cnt, shl_q;
11    sc_out<sc_logic> ser_q, sel_shift, sel_sub;
12
13    enum states {IDLE, INIT, LOAD, DIVIDE};
14    sc_signal<states> p_state, n_state;
15 }
```

Fig.1. state machine of unsigned divider and implementation in SystemC

FSM transitions through defined states (IDLE, INIT, LOAD, and DIVIDE) to coordinate the datapath operations over multiple clock cycles. A hybrid Mealy/Moore architecture is adopted for the output logic. While state transitions depend solely on the current state and specific status flags (such as cnt_comp), the control signals directed to the datapath (e.g., sel_sub, shl_q) utilize Mealy-type dependencies (like signbit) to execute register updates efficiently within a single clock cycle. Upon completion of the required iterations, the done signal is asserted, indicating that the stable quotient and remainder are available at the outputs.

```

16 SC_CTOR(controller) {
17
18     SC_METHOD(comb_nextstate_function);
19     sensitive << p_state << start << cnt_comp ;
20
21     SC_METHOD(comb_output_function);
22     sensitive << p_state << start << cnt_comp << signbit;
23
24     SC_THREAD(seq_function);
25     sensitive << clk << rst;
26 };
27 void comb_nextstate_function();
28 void comb_output_function();
29 void seq_function();
30 };

```

Fig.2. methods of controller

The finite state machine (FSM) of the division unit is designed based on the Huffman standard model, which explicitly separates sequential elements from combinational logic.

Sequential State Register (seq_function): This synchronous process is strictly sensitive to the positive edge of the clock signal and asynchronous reset. It is responsible for updating the current state (p_state) to the evaluated next state (n_state).

Combinational Next-State Logic (comb_nextstate_function): This combinational process evaluates the upcoming state transitions based on the current state and external status signals (e.g., start and cnt_comp).

Combinational Output Logic (comb_output_function): This combinational block generates the required control signals for the datapath.

```

3 void controller::comb_output_function() {
4
5     done          = SC_LOGIC_0;
6     clr_counter   = SC_LOGIC_0;
7     init_r        = SC_LOGIC_0;
8     load_d        = SC_LOGIC_0;
9     load_q        = SC_LOGIC_0;
10    load_r        = SC_LOGIC_0;
11    en_cnt         = SC_LOGIC_0;
12    shl_q          = SC_LOGIC_0;
13    ser_q          = SC_LOGIC_0;
14    sel_shift      = SC_LOGIC_0;
15    sel_sub        = SC_LOGIC_0;
16

```

Fig.3. inactive value of output

At the beginning of the comb_output_function, all control signals are explicitly driven to their inactive logic levels (SC_LOGIC_0). Following these default initializations, a switch-case statement evaluates the active state (p_state).

The operational flow of the restoring division unit is governed by a four-state finite state machine.

In the idle state, the done flag is asserted (SC_LOGIC_1), indicating that the unit is ready for a new operation or has

successfully completed the previous one. The FSM continuously monitors the start signal. Upon detecting a logic high, the controller transitions to the INIT state.

Once the start signal is acknowledged, the FSM enters the INIT state to prepare the datapath for execution. The counter is cleared, and the remainder register is initialized to 0. The done signal is explicitly de-asserted to indicate that the unit is busy. To prevent multiple unintended executions from a single long pulse, the FSM implements a handshaking mechanism; it remains in INIT until the start signal is de-asserted, after which it proceeds to the LOAD state.

The controller commands the datapath to sample the input operands by asserting load_d (for the divisor) and load_q (for the dividend). Since operand loading is completed in a single cycle, the FSM transitions unconditionally to the DIVIDE state in the subsequent clock cycle.

```

switch (p_state) {
case IDLE:
    done = SC_LOGIC_1;
    break;
case INIT:
    clr_counter = SC_LOGIC_1;
    init_r      = SC_LOGIC_1;
    done        = SC_LOGIC_0;
    break;
case LOAD:
    load_d = SC_LOGIC_1;
    load_q = SC_LOGIC_1;
    break;
}

void controller::comb_nextstate_function() {
    n_state = IDLE;
    switch (p_state) {
    case IDLE:
        if (start == SC_LOGIC_1)
            n_state = INIT;
        else
            n_state = IDLE;
        break;
    case INIT:
        if (start == SC_LOGIC_0)
            n_state = LOAD;
        else
            n_state = INIT;
        break;
    case LOAD:
        n_state = DIVIDE;
        break;
}

```

Fig.4. detail of each od states(idle, init, load)

divide state represents the core execution phase of the restoring division algorithm. The iteration counter is enabled, and the dividend register is commanded to shift left.

Restoring Condition (signbit == SC_LOGIC_1): If the subtraction yields a negative result, the remainder must be restored. The controller selects the shifted value (sel_shift = SC_LOGIC_1, sel_sub = SC_LOGIC_0) and shifts a '0' into the quotient register (ser_q = SC_LOGIC_0).

Non-Restoring Condition (signbit == SC_LOGIC_0): If the result is positive, the subtraction is valid. The controller updates the remainder with the subtracted value (sel_sub = SC_LOGIC_1, sel_shift = SC_LOGIC_0) and shifts a '1' into the quotient register (ser_q = SC_LOGIC_1).

The FSM remains in the DIVIDE state, repeating this process until the iteration counter reaches the specified parameter limit. Once the counter asserts the cnt_comp flag (cnt_comp == SC_LOGIC_1), the division is complete, and the FSM loops back to the IDLE state.

```

case DIVIDE:
    if (cnt_comp == SC_LOGIC_1)
        n_state = IDLE;
    else
        n_state = DIVIDE;
    break;

```

```

case DIVIDE:
    en_cnt = SC_LOGIC_1;
    shl_q = SC_LOGIC_1;
    load_r = SC_LOGIC_1;

    if (signbit == SC_LOGIC_1) {
        ser_q = SC_LOGIC_0;
        sel_shift = SC_LOGIC_1;
        sel_sub = SC_LOGIC_0;
    }
    else {
        ser_q = SC_LOGIC_1;
        sel_shift = SC_LOGIC_0;
        sel_sub = SC_LOGIC_1;
    }
    break;

```

Fig.5. divide state

The state register of the Huffman-style FSM is implemented within the seq_function routine. This procedural block continuously monitors the asynchronous reset and clock signals. Upon the assertion of the reset signal, the present state is immediately forced to the state to ensure a known initial condition. During normal operation, on every rising edge of the clock, the present state register is updated with the evaluated next state .

```

void controller::seq_function() {
    while(1){
        if (rst == '1')
            p_state = IDLE;
        else if (clk->event() && clk == '1')
            p_state = n_state;
        wait();
    }
}

```

Fig.6. sequential function of controler

B. Design of datapathUnsigned Restoring Division

The datapath of the restoring division unit is implemented as a structural SystemC module (Datapath), strictly separating the computational elements from the control logic. It executes the arithmetic operations dictated by the FSM control signals. Three primary registers manage the operands during the division process. The Divisor Register (DReg) holds the divisor loaded from the divisorBus. It remains static throughout the iteration phase. The Quotient/Dividend Register (QReg) initially captures the dividend. During the DIVIDE state, it acts as a shift register (ShIQ), shifting left to feed its Most Significant Bit (MSB) into

the remainder logic, while simultaneously receiving the calculated quotient bits serially via the SerQ input. The Remainder Register (RReg) is initialized to zero before the division starts. It acts as an accumulator, updating its value at each clock cycle based on the result of the trial subtraction.

```

template <int N>
class DRegister : public sc_module {
public:
    sc_in<sc_logic> clk, rst, load;
    sc_in<sc_lv<N>> regin;
    sc_out<sc_lv<N>> regout;

    SC_HAS_PROCESS(DRegister);
    DRegister(sc_module_name);

    void registering();
};

template <int N>
void DRegister<N>::registering()
{
    sc_lv<N> tmp;
    if (rst == '1')
    {
        for (int i = 0; i < N; i++)
            tmp[i] = sc_logic(0);
        regout = tmp;
    }
    else if ((clk->event()) && (clk == '1')) {
        if (load == '1')
            regout = regin;
    }
}

```

Fig.7. implementation of Dregister in SystemC

```

3 template <int N>
4 class QRegister : public sc_module {
5 public:
6     sc_in<sc_logic> clk, rst, load, shl, ser_in;
7     sc_in<sc_lv<N>> regin;
8     sc_out<sc_lv<N>> regout;
9
10    SC_HAS_PROCESS(QRegister);
11    QRegister(sc_module_name);
12
13    void registering();
14 };

template <int N>
void QRegister<N>::registering()
{
    sc_lv<N> tmp;
    if (rst == '1')
    {
        for (int i = 0; i < N; i++)
            tmp[i] = sc_logic(0);
        regout = tmp;
    }
    else if ((clk->event()) && (clk == '1')) {
        if (load == '1') regout = regin;
        else if (shl == '1')
            regout = (regout.read().range(N-2, 0), ser_in);
    }
}

```

Fig.8. implementation of Qregister in SystemC

```

template <int N>
void RRegister<N>::registering()
{
    sc_lv<N> tmp;
    if (rst == '1')
    {
        for (int i = 0; i < N; i++)
            tmp[i] = sc_logic(0);
        regout = tmp;
    }
    else if ((clk->event()) && (clk == '1')) {
        if (init == '1')
        {
            for (int i = 0; i < N; i++)
                tmp[i] = sc_logic(0);
            regout = tmp;
        }
        else if (load == '1')
            regout = regin;
    }
}

```

Fig.9. implementation of Rregister in SystemC

The core computational loop relies on a left shifter, a subtractor, and a multiplexer.

In each iteration, the current remainder and the MSB of the QReg are concatenated and shifted left by the LShifter module. The Subtractor (Sub) then performs subtraction between this shifted value and the divisor (DReg).

The sign bit of the subtraction result (signbit) is routed directly to the FSM controller to determine the next action. Based on this sign bit, the FSM controls the Multiplexer (Mux) via the SelSub signal to select either the restored value (the output of the shifter) or the non-restored value (the output of the subtractor) to be loaded back into the RReg.

```

template <int T>
SC_MODULE (Subtractor){

    sc_in< sc_lv<T> > in1, in2;
    sc_out< sc_lv<T> > out;
    sc_out< sc_logic> signbit;

    SC_CTOR (Subtractor)
    {
        SC_METHOD(Subtracting);
        sensitive << in1 << in2;
    }

    void Subtracting()
    {
        sc_int<T-1> diff;
        sc_lv<T> vout;
        sc_logic vsign;

        diff = (sc_uint<T>) in1 - (sc_uint<T>) in2;
        vout = diff.range(T-1, 0);

        if (diff < 0)
            vsign = SC_LOGIC_1;
        else
            vsign = SC_LOGIC_0;

        out = vout;
        signbit = vsign;
    }
};

```

Fig.10. implementation of subtract module

```

template <int N>
class LeftShifter : public sc_module {
public:

    sc_in<sc_lv<N>> in_data;
    sc_in<sc_logic> ser_in;
    sc_out<sc_lv<N>> out_data;

    SC_CTOR(LeftShifter)
    {
        SC_METHOD(shifting);
        sensitive << in_data << ser_in;
    }

    void shifting();
};

```

```

template <int N>
void LeftShifter<N>::shifting()
{
    out_data = (in_data.read()).range(N-2, 0), ser_in.read();
}

```

Fig.11. implementation of Leftshifter

```

template <int W = 8>
class Mux2to1 : public sc_module {
public:
    sc_in<sc_logic> sel;
    sc_in<sc_lv<W>> in0, in1;
    sc_out<sc_lv<W>> out;

    SC_CTOR(Mux2to1)
    {
        SC_METHOD(muxing);
        sensitive << in0 << in1 << sel;
    }

    void muxing();
};

template <int W>
void Mux2to1<W>::muxing() {
    if (sel == '1') out= in1;
    else out=in0;
}

```

Fig.12. implementation of multiplexer

To manage the loop execution, synchronous counter (Cnt) is integrated within the datapath. The counter increments under the control of the enCnt signal. A concurrent SystemC method (datapath_assign) continuously monitors the counter's output. Once the counter reaches 15 (representing 16 iterations for a 16-bit division), the combinational logic asserts the cnt_comp flag to signal the FSM that the division process is complete.

The template parameter N defines the bit-width of the counter output, which defaults to a width of 4 bits. The module interface consists of standard control inputs of type sc_logic, including the clock, asynchronous reset, a synchronous clear and an active-high enable signal.

The core counting logic is encapsulated in the counting member function. In the module constructor, this function is registered as an SC_METHOD, with its sensitivity list explicitly tied to the clock and reset signals. The internal logic prioritizes the asynchronous reset, which clears the output vector unconditionally. On a valid rising clock edge, the synchronous clr signal takes precedence over the counting operation. If the module is not being cleared and the enable flag is asserted, the current value of the cnt_out port is read, converted to an unsigned integer type for arithmetic manipulation, incremented by one, and subsequently written back to the output port.

```

template <int N = 4>
class Counter : public sc_module {
public:

    sc_in<sc_logic> clk, rst;
    sc_in<sc_logic> clr, en;
    sc_out<sc_lv<N>> cnt_out;

    SC_CTOR(Counter)
    {
        SC_METHOD(counting);
        sensitive << rst << clk;
    }

    void counting();
};

template <int N>
void Counter<N>::counting()
{
    if (rst == '1') cnt_out = 0;
    else if (clk.event() && (clk == '1')) {
        if (clr.read() == '1') cnt_out = 0;
        else if (en == '1')
            cnt_out = cnt_out.read().to_uint() + 1;
    }
}

```

Fig.13. implementation of counter in systemC

As illustrated in Fig. 14, the datapath is implemented as a top-level structural module within the SystemC environment, seamlessly interconnecting several parameterized sub-components to form the restoring division architecture. The hardware realization is achieved by instantiating dedicated 16-bit storage elements—namely the divisor register (DReg), the quotient register (QReg), and the remainder register (RReg).

In addition to the registers, the datapath incorporates instantiated combinational logic blocks essential for the arithmetic operations and data routing. These include a left shifter (LShifter), a subtractor (Sub), and a 2-to-1 multiplexer (Mux). To manage the iterative nature of the algorithm independently from the main FSM states, a 4-bit synchronous counter (Cnt) is also instantiated to monitor the execution cycles.

The structural interconnections among these instances are established using internal SystemC signals, such as shift_out, sub_out, and mux_out, which act as internal buses. Furthermore, a continuous assignment mechanism is realized through the concurrent datapath_assign method. This method is strictly sensitive to register changes and is responsible for routing the final outputs to RBus and QBus, extracting the

Most Significant Bit (MSB) of the quotient register (Qregister_msb), and dynamically evaluating the counter's state to assert the cnt_comp flag once 151515 iterations are completed.

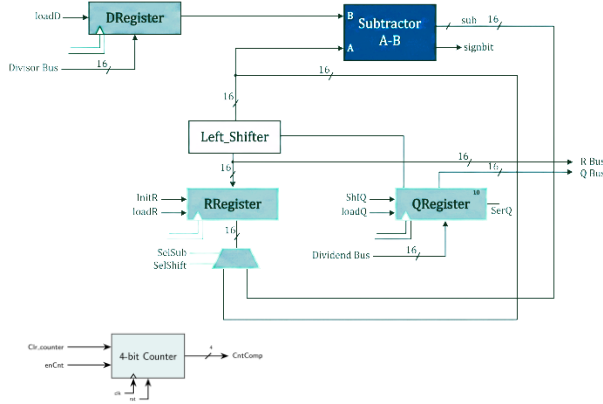


Fig.14. datapath of unsigned divider

```
SC_MODULE(Datapath) {
    sc_in<sc_logic> clk, rst;
    sc_in<sc_logic> loadD;
    sc_in<sc_logic> InitR, loadR;
    sc_in<sc_logic> SelSub;
    sc_in<sc_logic> ShIQ, loadQ, SerQ;
    sc_in<sc_logic> clrCnt, enCnt;
    sc_in<sc_lv<16>> divisorBus, dividendBus;

    sc_out<sc_logic> signbit, cnt_comp;
    sc_out<sc_lv<16>> RBus, QBus;
    sc_out<sc_lv<4>> count_out;
}
```

Fig.15. the ports of input and output of datapath

C. Top-Level System Integration

The complete architecture of the restoring division unit is realized by integrating the datapath and the Finite State Machine (FSM) controller into a unified top-level SystemC module, designated as DividerTop.

At the interface level, the system interacts with external testbenches or higher-level modules through primary input and output ports. The inputs include global synchronization signals (clk and rst), a start command to initiate the process, and two 16-bit buses for the divisor and dividend (divisorBus and dividendBus). The system outputs the final remainder and quotient via RBus and QBus, respectively, alongside a ready flag that indicates the successful completion of the division algorithm.

A dedicated set of internal SystemC signals (sc_signal) is defined to route the control words and status flags between these two instances. The controller dictates the behavior of the datapath by issuing specific command signals to manage

register updates and arithmetic operations. Conversely, the datapath feeds critical status information back to the controller. This feedback loop primarily consists of the signbit, which dictates the trial subtraction outcome, and the cnt_comp flag, which alerts the FSM when the counter reaches 15, signaling the end of the iterative division cycle.

```
#include <systemc.h>
#include "datapath_divider_unsigned.h"
#include "controller_divider_unsigned.h"

SC_MODULE(DividerTop) {
    sc_in<sc_logic> clk, rst, start;
    sc_in<sc_lv<16>> divisorBus, dividendBus;

    sc_out<sc_lv<16>> RBus, QBus;
    sc_out<sc_logic> ready;

    sc_signal<sc_logic> load_d, init_r, load_r;
    sc_signal<sc_logic> sel_sub, shl_q, load_q, ser_q, sel_shift;
    sc_signal<sc_logic> clr_counter, en_cnt;
    sc_signal<sc_logic> signbit, cnt_comp;
    sc_signal<sc_lv<4>> count_out;

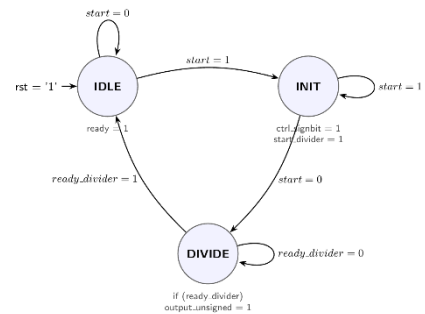
    Datapath* DP;
    controller* CNTRL;
}
```

Fig.16. design of top module in unsigned divider

D. Signed Division Extension and Wrapper Design

The controller for the signed division wrapper is implemented using a robust, three-process Huffman finite state machine (FSM) architecture modeled in SystemC. The design strictly separates sequential state updates from combinational logic, ensuring a latch-free and predictable hardware inference.

Based on the provided RTL code and the state diagram, the controller utilizes three distinct operational states: IDLE, INIT, and DIVIDE. The functionality is divided into three dedicated SystemC processes:



```
SC_MODULE(Controller) {
    sc_in<sc_logic> clk, rst, start;
    sc_in<sc_logic> ready_divider;

    sc_out<sc_logic> ready;
    sc_out<sc_logic> start_divider;
    sc_out<sc_logic> ctrl_signbit;
    sc_out<sc_logic> output_unsigned;

    enum states { IDLE, INIT, DIVIDE };
    sc_signal<states> p_state, n_state;
}
```

Fig.17. state diagram of controller

Seq_function method is implemented as an SC_THREAD (sensitive to the clock) and reset signals. It utilizes an infinite while(1) loop accompanied by a wait() statement to synchronize with the SystemC simulation kernel. The logic features an asynchronous active-high reset. When rst='1', the present state is immediately forced to IDLE.

```
void Controller::seq_function() {
    while(1){
        if (rst == '1')
            p_state = IDLE;
        else if (clk->event() && clk == '1')
            p_state = n_state;
        wait();
    }
}
```

Fig.19. sequential function of controller

Combinational Next-State Logic (comb_nextstate_function) as an SC_METHOD, this block determines the state transitions and is sensitive to p_state, start, and ready.

In the IDLE state, the machine waits for the initiation signal. If start='1', it transitions to INIT. In the INIT state, the controller employs a handshake mechanism, holding until the input stimulus drops (start='0') before proceeding to the DIVIDE state. In the DIVIDE state, the FSM monitors the unsigned division core. It remains in this state until ready_divider='1', after which it cycles back to IDLE.

```
void Controller::comb_nextstate_function() {
    n_state = IDLE;

    switch (p_state) {
        case IDLE:
            if (start == SC_LOGIC_1)
                n_state = INIT;
            else
                n_state = IDLE;
            break;

        case INIT:
            if (start == SC_LOGIC_0)
                n_state = DIVIDE;
            else
                n_state = INIT;
            break;

        case DIVIDE:
            if (ready_divider == SC_LOGIC_1)
                n_state = IDLE;
            else
                n_state = DIVIDE;
            break;
    }
}
```

Fig.20. Combinational Next-State Logic

Combinational Output Logic (comb_output_function) is also an SC_METHOD, sensitive to p_state and ready_divider. Similar to the next-state logic, all outgoing control signals are initially driven to SC_LOGIC_0. This initial default assignment completely eliminates the risk of implied memory (latches) across different states.

The control signals are predominantly designed as Moore outputs, meaning they depend exclusively on the current state. However, the system exhibits a hybrid Mealy-type behavior in the DIVIDE state: the output_unsigned signal conditionally evaluates to SC_LOGIC_1 only when the datapath core asserts ready_divider='1'.

```
void Controller::comb_output_function() {
    ready = SC_LOGIC_0;
    start_divider = SC_LOGIC_0;
    ctrl_signbit = SC_LOGIC_0;
    output_unsigned = SC_LOGIC_0;

    switch (p_state) {
        case IDLE:
            ready = SC_LOGIC_1;
            break;

        case INIT:
            ctrl_signbit = SC_LOGIC_1;
            start_divider = SC_LOGIC_1;
            break;

        case DIVIDE:
            if (ready_divider == SC_LOGIC_1)
                output_unsigned = SC_LOGIC_1;
            else
                output_unsigned = SC_LOGIC_0;
            break;
    }
}
```

Fig.21. Combinational Output Logic

```
SC_CTOR(Controller) {
    SC_METHOD(comb_nextstate_function);
    sensitive << p_state << start << ready_divider;

    SC_METHOD(comb_output_function);
    sensitive << p_state << ready_divider;

    SC_THREAD(seq_function);
    sensitive << clk << rst;
};

void comb_nextstate_function();
void comb_output_function();
void seq_function();
```

Fig.22. methods of controller

The datapath of the signed division wrapper is implemented as a structural model in SystemC. It acts as an interface layer around the existing unsigned divider core, ensuring that negative operands are properly converted to their absolute magnitudes before division, and the resulting outputs are assigned the correct mathematical signs.

To achieve this hardware behavior, several fundamental digital components were designed and instantiated within the main WrapperDatapath module. These sub-components

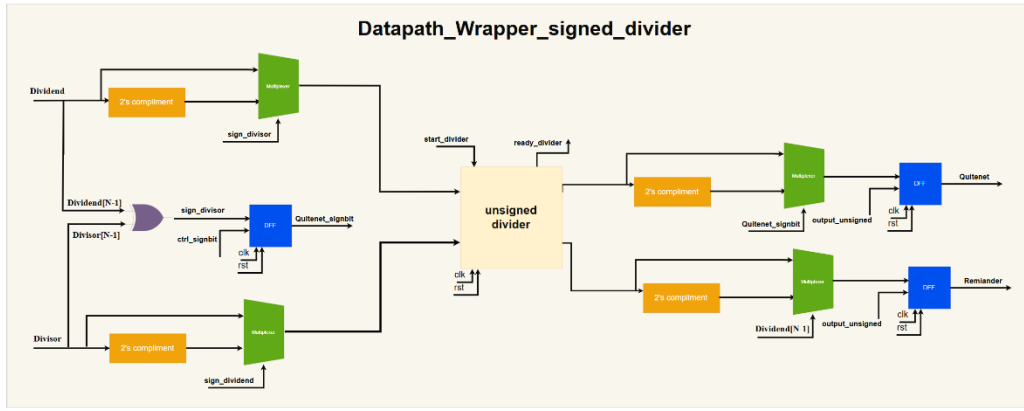


Fig.23. datapath of wrapper

include N-bit two's complementers, two-to-one multiplexers (MUX), a two-input Exclusive OR (XOR) gate, and D-type flip-flops (FF) for data synchronization. First, the sign bits, which correspond to the Most Significant Bit (MSB) of the operands, are extracted continuously via the `extract_signs` method. The sign of the final quotient is determined by evaluating the sign bits of the dividend and the divisor. This register is enabled by the `ctrl_signbit` control signal, preserving the sign information throughout the multi-cycle execution of the core divider.

During the pre-processing stage, the input operands must be converted to absolute magnitudes. The input data is fed into a `TwosComplement` module to generate its negative representation. A MUX (multiplexer_dividend and multiplexer_divisor) is then utilized to select between the original operand and its complemented version. The selection line for each MUX is directly tied to the respective operand's sign bit. Consequently, negative inputs are automatically inverted to positive values before being routed to the core buses.

The central component, `DividerTop`, is instantiated to perform the standard restoring division algorithm on the absolute magnitudes. It receives the prepared positive operands and is controlled by the `start_divider` signal.

Once the core algorithm concludes and asserts the `ready_divider` flag, the post-processing stage initiates to adjust the signs of the raw outputs. Similar to the input stage, the unsigned quotient and remainder from the core are routed through parallel two's complementers. A dedicated MUX (multiplexer_quotient) uses the previously stored `Sq` from the sign F as its selection line to output the correctly signed quotient. Conversely, according to the arithmetic rules of division, the remainder's sign strictly follows the original sign of the dividend. Therefore, multiplexer_remainder is controlled directly by dividend

Finally, the adjusted quotient and remainder vectors are captured in N-bit output FFs (`d_flipflop_quotient` and `d_flipflop_remainder`). These output registers are controlled by the `output_unsigned` signal provided by the wrapper's state machine, ensuring that the external output buses are updated precisely when the correct, finalized data is ready to be sampled.

E. Top module of wrapper

The top-level module, `SignedDividerTop`, serves as the highest hierarchical level of the signed division system. Its primary purpose is to structurally encapsulate the previously defined datapath and controller units, merging them into a single, cohesive, and fully functional hardware component. The module is implemented using a template structure (template `<int N>`), which parametrizes the bit-width of the operands and allows for scalable Register Transfer Level (RTL) design without requiring hard-coded modifications.

The external interface of the module consists of standard control pins and data buses. The input ports include a system clock (`clk`), an active-high asynchronous reset (`rst`), and an initiation trigger (`start`), alongside the N-bit signed dividend and divisor vectors. The output ports provide the calculated N-bit quotient and remainder vectors, as well as a ready flag to indicate the completion of the division cycle to external systems.

To facilitate internal communication between the control unit and the operational logic. These internal buses include `sig_start_divider`, `sig_ctrl_signbit`, and `sig_output_unsigned`, which transmit execution commands from the state machine to the datapath components. Additionally, the `sig_ready_divider` signal functions as a feedback loop, routing the operational status from the core divider back to the wrapper controller.

During the instantiation phase within the module's constructor (SC_CTOR), the WrapperDatapath and Controller sub-modules are dynamically allocated and structurally mapped. The external input and output ports are directly bound to their corresponding counterparts in the sub-modules. Meanwhile, the internal sc_signal wires are utilized to interconnect the respective control and status pins of the datapath and controller. This strict separation of data manipulation and state control adheres to standard digital design methodologies, ensuring that the module remains highly modular and easy to verify during simulation.

```
#include <systemc.h>
#include "wrapper_divider_datapath.h"
#include "wrapper_divider_signed_controller.h"

template <int N = 16>
SC_MODULE(SignedDividerTop) {
    sc_in<sc_logic> clk;
    sc_in<sc_logic> rst;
    sc_in<sc_logic> start;
    sc_in<sc_lv<N>> dividend;
    sc_in<sc_lv<N>> divisor;

    sc_out<sc_logic> ready;
    sc_out<sc_lv<N>> quotient;
    sc_out<sc_lv<N>> remainder;

    sc_signal<sc_logic> sig_ready_divider;
    sc_signal<sc_logic> sig_start_divider;
    sc_signal<sc_logic> sig_ctrl_signbit;
    sc_signal<sc_logic> sig_output_unsigned;
```

Fig.24. top module of signed divider

F. Bus-Functional Modeling of the Signed Divider

In the third phase of the design process, the architectural abstraction level was elevated from a detailed Register Transfer Level (RTL) structural model to a Bus-Functional Model (BFM). The primary goal of this transition is to encapsulate the signed division functionality using a transaction-centric approach. As seen in the provided SystemC implementation, the divider<W, NumClk> template module maintains pin-to-pin compatibility with the previous RTL design. It uses identical interface ports: input buses for Dividend and Divisor, control signals (clk, rst, start), output buses for Quotient and Remainder, and the ready handshake flag. This ensures that the BFM can seamlessly replace the RTL module in a larger system simulation without requiring modifications to the external environment.

Instead of executing cycle-by-cycle register updates and microarchitectural state machine transitions, the BFM abstractly models the elapsed time and communication protocols. Inside the operation thread, when the start signal is sampled high, the module immediately lowers the ready flag. To accurately reflect the transient state of the hardware during computation, the output buses are driven to an unknown state (SC_LOGIC_X).

```
#include <iostream>
#include <systemc.h>

template <int W, int NumClk>
SC_MODULE(divider)
{
    sc_in<sc_lv<W>> Dividend, Divisor;
    sc_in<sc_logic> clk, rst, start;

    sc_out<sc_lv<W>> Quotient, Remainder ;
    sc_out<sc_logic> ready;

    SC_CTOR(divider)
    {
        SC_THREAD(operation);
        sensitive << clk << rst;
    }
    void operation() ;
};
```

Fig.25. bus functional model of divider

The core of the behavioral abstraction lies in the “timing mimicry” mechanism. In a structural RTL divider, a shift-and-subtract algorithm naturally consumes multiple clock cycles to process data bit by bit. To replicate this delay without simulating the underlying gates, the BFM utilizes a parameter NumClk. A simple for loop iterates NumClk times, calling wait(clk->posedge_event()) in each iteration, forcing the module to stall for the exact number of clock cycles that the real hardware would require. Consequently, external components observing the BFM experience the exact same timing latency and protocol handshakes as they would with the RTL implementation.

```
else if ((clk == '1' && (clk->event()) )
{
    if (start == '1')
    {
        ready= SC_LOGIC_0;

        sc_lv<W> unknown_val;
        for(int k=0; k<W; k++) unknown_val[k] = SC_LOGIC_X;
        Quotient= unknown_val;
        Remainder= unknown_val;

        // Mimicking hardware clock cycles delay
        for (int i = 0; i < NumClk; i++)
            wait(clk->posedge_event());

        ready= SC_LOGIC_1;

        if (Divisor.read() != 0) {
            Q_var = sc_int<W>(Dividend.read()) / sc_int<W>(Divisor.read());
            Quotient = Q_var;
            Remainder = sc_int<W>(Dividend.read()) - (sc_int<W>(Divisor.read()) * Q_var);
        }
        else {
            Q_var = 0;
            Quotient = Q_var;
            Remainder = sc_int<W>(Dividend.read()) - (sc_int<W>(Divisor.read()) * Q_var);
        }
    }
}
```

Fig.26. design of bfm and mimic hardware clock cycle

the BFM asserts the ready flag and computes the division results. Rather than utilizing bitwise operations, the model leverages standard C++ arithmetic operators over the sc_int<W> datatype to handle the signed operations natively. The logic includes a safeguard against division-by-zero errors. If the divisor is valid, the quotient is calculated directly via the / operator, and the remainder is derived algebraically based on the relation Remainder=Dividend-(Divisor×Quotient).

E. simulation and result

To validate the operational integrity of the hardware division logic SystemC testbench (DividerTB) was developed. The primary objective of this verification environment is to stimulate the Design Under Test (DUT) with pseudo-random inputs and cross-verify the hardware outputs against a software-based “golden model.”

The testbench architecture is driven by two main parallel threads:

Clock Generation (clocking): This thread generates a continuous system clock with a period of 100 ns, establishing a synchronous baseline for the sequential logic within the DUT.

Automated Stimulus and Verification (testing): Rather than relying on hard-coded manual test vectors, this thread implements a randomized verification strategy. For a predefined number of iterations.

```
SC_MODULE(DividerTB) {

    sc_signal<sc_logic> clk, rst, start;
    sc_signal<sc_lv<16>> divisorBus, dividendBus;
    sc_signal<sc_lv<16>> RBus, QBus;
    sc_signal<sc_logic> ready;

    DividerTop* div_top;

    SC_CTOR(DividerTB) {

        div_top = new DividerTop("Divider_1");
        div_top->clk(clk);
        div_top->rst(rst);
        div_top->start(start);
        div_top->divisorBus(divisorBus);
        div_top->dividendBus(dividendBus);
        div_top->RBus(RBus);
        div_top->QBus(QBus);
        div_top->ready(ready);

        SC_THREAD(testing);
        sensitive << clk.posedge_event();

        SC_THREAD(clocking);
    }

    void clocking();
    void testing();
};
```

Fig.25. testbench of unsigned divider

The testbench architecture is driven by two main parallel threads:

Initialization: A hardware reset is triggered by pulsing the rst signal to ensure the internal state machine of the divider starts from a known, idle condition.

Stimulus Generation: Two pseudo-random 16-bit integers are generated. To prevent invalid arithmetic operations (such as division by zero), the divisor is strictly constrained to a non-zero range. The generated values are applied to the dividendBus and divisorBus, followed by a single-cycle pulse on the start signal.

Synchronization: The testbench suspends its execution thread using wait(ready.posedge_event()), effectively stalling until the DUT asserts the ready flag, indicating the completion of the division cycle.

Self-Checking Mechanism: Upon completion, the testbench reads the hardware-calculated results from QBus and RBus. Simultaneously, it computes the expected theoretical results using standard C++ arithmetic operators. The hardware outputs are then strictly compared against these expected values. Any discrepancies trigger an “ERROR” log in the console, while matches output a “SUCCESS” message alongside the timestamp and computed values. Once all iterations conclude, sc_stop() is invoked to gracefully terminate the simulation kernel.

```
Info: (I702) default timescale unit used for tracing: 1
@ 2050 ns : SUCCESS! 45298 / 35005 = 1 (R: 10293)
@ 4250 ns : SUCCESS! 64677 / 5340 = 12 (R: 597)
@ 6450 ns : SUCCESS! 20738 / 36754 = 0 (R: 20738)
@ 8650 ns : SUCCESS! 48946 / 7577 = 6 (R: 3484)
@ 10850 ns : SUCCESS! 17079 / 2720 = 6 (R: 759)
@ 13050 ns : SUCCESS! 38357 / 30890 = 1 (R: 7467)
@ 15250 ns : SUCCESS! 28835 / 41578 = 0 (R: 28835)
@ 17450 ns : SUCCESS! 9158 / 52179 = 0 (R: 9158)
@ 19650 ns : SUCCESS! 20955 / 6757 = 3 (R: 684)
@ 21850 ns : SUCCESS! 6772 / 50011 = 0 (R: 6772)
```

Fig.26. computation of values in unsigned divider

The sc_main function serves as the primary entry point for the SystemC simulation environment. Within this module, the testbench instance DividerTB is instantiated to drive the simulation. To facilitate post-simulation analysis, a Value Change Dump (VCD) file named Divider_Waveform is initialized using sc_create_vcd_trace_file. Key signals, including the clock (clk), reset (rst), handshaking controls (start and ready), and data buses (divisorBus, dividendBus, QBus, and RBus), are registered for tracing.

```
int sc_main(int argc, char **argv) {

    DividerTB TB1("Testbench_Top");

    sc_trace_file* vcdfile;
    vcdfile = sc_create_vcd_trace_file("Divider_Waveform");

    sc_trace(vcdfile, TB1.clk, "clk");
    sc_trace(vcdfile, TB1.rst, "rst");
    sc_trace(vcdfile, TB1.start, "start");
    sc_trace(vcdfile, TB1.ready, "ready");
    sc_trace(vcdfile, TB1.divisorBus, "divisorBus");
    sc_trace(vcdfile, TB1.dividendBus, "dividendBus");
    sc_trace(vcdfile, TB1.QBus, "QBus");
    sc_trace(vcdfile, TB1.RBus, "RBus");
}
```

Fig.27. main of unsigned divider

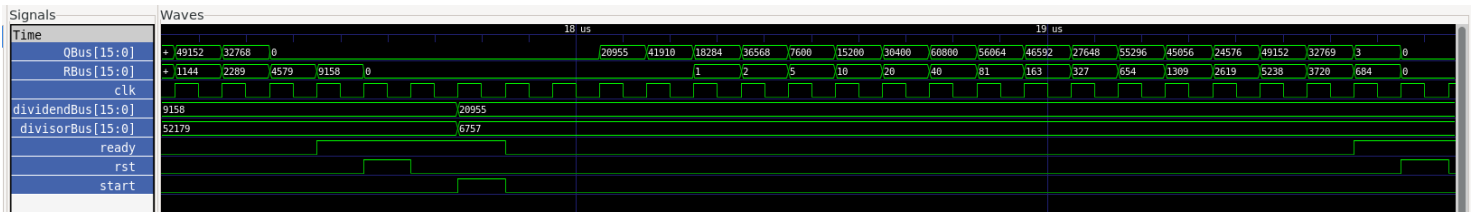


Fig.28. waveform of unsigned divider

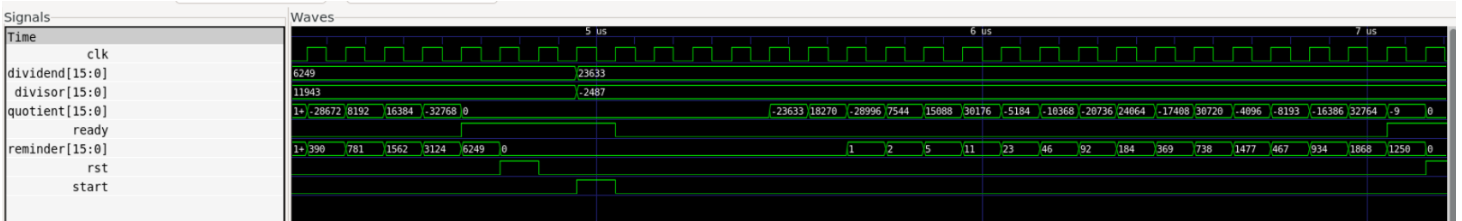


Fig.30. waveform of signed divider

Following the simulation, the generated VCD file was imported into GTKWave for visual inspection. The waveform graphical interface confirms the correct chronological behavior of the module. Specifically, the active-high reset initializes the internal states, and the ready signal correctly drops upon the assertion of the start pulse. The data tracking shows that the inputs on divisorBus and dividendBus are accurately latched, and after the designated clock cycles, the calculated quotient and remainder appear correctly on QBus and RBus. The waveform results perfectly align with the expected software calculations, verifying the structural and functional integrity of the design.

The SignedDividerTB module automates the verification process for the signed division hardware. To thoroughly test the absolute value converters and sign extraction logic, the testbench must feed both positive and negative integers into the datapath. The stimulus generator achieves this by using the standard C++ rand() function.

```
#include <systemc.h>
#include "wrapper_divider_design.h"
SC_MODULE(SignedDividerTB) {
    sc_signal<sc_logic> clk, rst, start, ready;
    sc_signal<sc_lv<16>> dividend, divisor, quotient, remainder;

    SignedDividerTop<16>* DividerSign;

    SC_CTOR(SignedDividerTB) {
        DividerSign = new SignedDividerTop<16>("Signed_Divider");
        DividerSign->clk(clk);
        DividerSign->rst(rst);
        DividerSign->start(start);
        DividerSign->dividend(dividend);
        DividerSign->divisor(divisor);
        DividerSign->ready(ready);
        DividerSign->quotient(quotient);
        DividerSign->remainder(remainder);

        SC_THREAD(testing);
        sensitive << clk.posedge_event();

        SC_THREAD(clocking);
    }
    void clocking();
    void testing();
};
```

Fig.29. testbench of signed divider

A specific check prevents division by zero by forcing the divisor to 1 if the random generator produces a zero. After asserting the start signal, the testbench waits for the hardware to assert the ready flag. Once ready goes high, the testbench reads the hardware outputs (quotient and remainder), casts them back to standard C++ integers, and compares them against the expected results computed using the native / and % operators. Any discrepancy between the SystemC datapath output and the software model is logged as an error with the exact simulation timestamp.

```
Info: (1702) default timescale unit used for tracing: 1 ps (Signed_Divider_Waveform.vcd)
@ 2250 ns : SUCCESS! -15001 / -10686 = 1 (R: -4315)
@ 4650 ns : SUCCESS! 6249 / 11943 = 0 (R: 6249)
@ 7050 ns : SUCCESS! 23633 / -2487 = -9 (R: 1250)
@ 9450 ns : SUCCESS! 5194 / 15170 = 0 (R: 5194)
@ 11850 ns : SUCCESS! -24791 / 17334 = -1 (R: -7457)
@ 14250 ns : SUCCESS! -10054 / -22485 = 0 (R: -10054)
@ 16650 ns : SUCCESS! -15886 / -8013 = 1 (R: -7873)
@ 19050 ns : SUCCESS! 10723 / -10611 = -1 (R: 112)
@ 21450 ns : SUCCESS! -32644 / 16019 = -2 (R: -606)
@ 23850 ns : SUCCESS! -30636 / -2646 = 11 (R: -1530)
```

Fig.31. result of computation of divider

The provided code is the fundamental entry point for the SystemC simulation environment. Within this block, the primary testbench module, SignedDividerTB, is instantiated under the identifier TB1. To effectively capture and analyze the dynamic behavior of the designed hardware, a Value Change Dump (VCD) trace file designated as "Signed_Divider_Waveform" is initialized. The sc_trace function is subsequently utilized to map essential control and data signals—such as the clock, reset, handshaking flags, and arithmetic operands—directly to this trace file. Ultimately, invoking the sc_start() command triggers the underlying SystemC simulation kernel to commence execution.

```
int sc_main(int argc, char **argv) {
    SignedDividerTB TB1("testbench1");

    sc_trace_file* vcdfile;
    vcdfile = sc_create_vcd_trace_file("Signed_Divider_Waveform");

    sc_trace(vcdfile, TB1.clk, "clk");
    sc_trace(vcdfile, TB1.rst, "rst");
    sc_trace(vcdfile, TB1.start, "start");
    sc_trace(vcdfile, TB1.ready, "ready");
    sc_trace(vcdfile, TB1.dividend, "dividend");
    sc_trace(vcdfile, TB1.divisor, "divisor");
    sc_trace(vcdfile, TB1.quotient, "quotient");
    sc_trace(vcdfile, TB1.remainder, "remainder");
```

Fig.31. main of wrapper

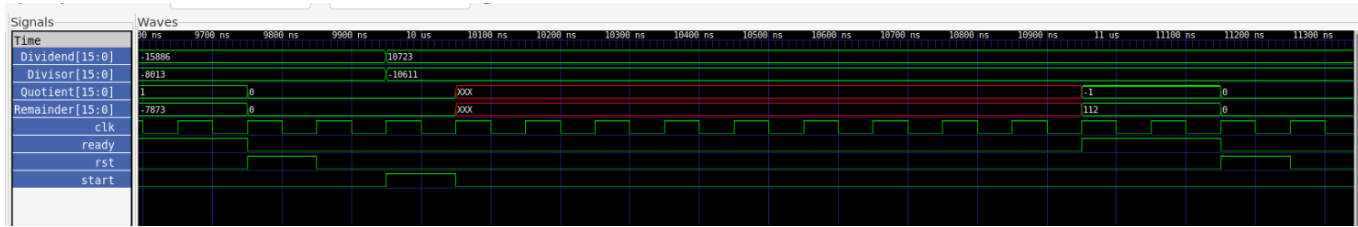


Fig.34. waveform of signed divider with bfm design

To comprehensively validate the bus-functional model (BFM) of the signed divider was developed using SystemC. The verification environment is structured around two concurrent processes: one dedicated to clock generation and another for stimulus application and response monitoring.

The clocking mechanism generates a symmetrical square wave with a 100 ns period, driving the synchronous elements of the testbench. Within the primary testing thread, a finite number of test cases are executed. For each iteration, a system reset is applied before generating randomized 16-bit signed integer values for the dividend and divisor. To prevent undefined behavior during testing, a simple software safeguard ensures the divisor is never zero.

Following the initialization of the operands, the testbench asserts the start signal and subsequently halts execution, polling the ready flag. This accurately tests the handshaking protocol and the latency defined within the BFM parameters. Once the ready signal goes high, the testbench captures the output Quotient and Remainder signals. These simulated outputs are cast back to standard integers and immediately compared against expected values computed using native C++ arithmetic operators. The testbench automatically logs a timestamped success or error message for each test vector, providing an automated and highly reliable verification mechanism.

```
SC_MODULE(dividerTB) {
    sc_signal<sc_logic> clk, rst, start, ready;
    sc_signal<sc_lv<16>> Dividend, Divisor, Quotient, Remainder;
    divider<16, 9>* DIV;
    SC_CTOR(dividerTB) {
        DIV = new divider<16, 9>("Divider_signed");
        DIV->Dividend(Dividend);
        DIV->Divisor(Divisor);
        DIV->clk(clk);
        DIV->rst(rst);
        DIV->start(start);
        DIV->Quotient(Quotient);
        DIV->Remainder(Remainder);
        DIV->ready(ready);

        SC_THREAD(testing);
        sensitive << clk.posedge_event();
        SC_THREAD(clocking);
    }
    void clocking();
    void testing();
};
```

Fig.32. testbench of bfm of divider

```
Info: (1702) default timescale unit used for tracing: 1 ps (Divider_Signed_Waveform.vcd)
@ 1250 ns : SUCCESS! -15001 / -10686 = 1 (R: -4315)
@ 2650 ns : SUCCESS! 6249 / 11943 = 0 (R: 6249)
@ 4050 ns : SUCCESS! 23633 / -2487 = -9 (R: 1250)
@ 5450 ns : SUCCESS! 5194 / 15170 = 0 (R: 5194)
@ 6850 ns : SUCCESS! -24791 / 17334 = -1 (R: -7457)
@ 8250 ns : SUCCESS! -10054 / -22485 = 0 (R: -10054)
@ 9650 ns : SUCCESS! -15886 / -8013 = 1 (R: -7873)
@ 11050 ns : SUCCESS! 10723 / -10611 = -1 (R: 112)
@ 12450 ns : SUCCESS! -32644 / 16019 = -2 (R: -606)
@ 13850 ns : SUCCESS! -30636 / -2646 = 11 (R: -1530)
```

Fig.33. result of computation in divider

To observe the dynamic behavior of the signed divider, the top-level simulation environment is configured within the sc_main entry point. The testbench module, divider_signed_TB, is instantiated to drive the test vectors and monitor the system. For detailed visual verification and timing analysis, a Value Change Dump (VCD) file named Divider_Signed_Waveform is initialized.

All primary interface signals are registered to this trace file. This includes the system clock and reset, the handshaking controls (start and ready), and the 16-bit arithmetic data buses (Dividend, Divisor, Quotient, and Remainder). The simulation is set to execute for a total duration of 500,000 ns, which provides ample time to process the loop of randomized test cases. Upon completion of the simulation, the trace file is explicitly closed to ensure all waveform data is properly flushed and saved without corruption. The resulting VCD file can then be imported into waveform viewers, such as GTKWave, to verify the cycle-accurate stalling and the exact timing of the handshake protocol.

```
int sc_main(int argc, char *argv[])
{
    dividerTB* TOP = new dividerTB("divider_signed_TB");
    sc_trace_file* VCDFile;
    VCDFile = sc_create_vcd_trace_file("Divider_Signed_Waveform");

    sc_trace(VCDFile, TOP->Dividend, "Dividend");
    sc_trace(VCDFile, TOP->Divisor, "Divisor");
    sc_trace(VCDFile, TOP->clk, "clk");
    sc_trace(VCDFile, TOP->rst, "rst");
    sc_trace(VCDFile, TOP->start, "start");
    sc_trace(VCDFile, TOP->ready, "ready");
    sc_trace(VCDFile, TOP->Quotient, "Quotient");
    sc_trace(VCDFile, TOP->Remainder, "Remainder");

    sc_start(500000, SC_NS);
    sc_close_vcd_trace_file(VCDFile);

    return 0;
}
```

Fig.34. main of bus function

The provided waveform illustrates the operational behavior of a Bus Functional Model (BFM), demonstrating its role as a bridge between high-level transactional interfaces and pin-level bus protocols. According to the foundational concepts of functional modeling, a BFM abstracts the internal microarchitecture of a target design while strictly maintaining cycle-accurate timing at its boundaries.

the BFM initiates a new transaction by sampling the input operands, denoted as Dividend and divisor. In a BFM methodology, the core arithmetic operation is not performed step-by-step through a structural datapath. Instead, the model evaluates the mathematical result functional and instantaneously at a higher abstraction level (e.g., using native C++ arithmetic). However, to seamlessly integrate with the rest of the hardware simulation, the BFM must exhibit cycle-accurate timing. This is achieved by intentionally mimicking the clock delay of the actual hardware implementation.

As explicitly annotated in the waveform, there is a deterministic processing delay of exactly 9 clock cycles ("12 clk") between the start command and the subsequent assertion of the ready signal. During this mimicked processing window, the output Result bus is explicitly driven to an unknown logic state, visually depicted by the red "X" traces. In the context of a BFM, this red zone does not represent ongoing internal logic transitions. Rather, it is a deliberate modeling artifact used to enforce the bus protocol. It acts as a strict indicator to downstream system components that the transaction is still in progress and any data currently present on the bus is invalid. Once the mimicked 9-cycle delay elapses, the ready signal transitions to high.